

Producer–Consumer: задача

Один поток (Producer) помещает элементы в общую очередь, другой поток (Consumer) извлекает их и обрабатывает. Очередь — общий буфер с конечной или бесконечной емкостью; правильно организованная синхронизация гарантирует отсутствие потерь, дублирования и корректный порядок обработки.

Проблема: гонки и простые мьютексы

Без синхронизации операции push/pop пересекаются — состояние очереди портится (коррупция указателей, неверный size).

Использование только mutex защищает критические секции, но если Consumer вызывает pop на пустой очереди, возникает busy-wait или постоянные проверки цикла, что расходует CPU и увеличивает задержки.



Mutex

Защищает структуру, но не решает ожидание события появления данных.

Busy-wait

Consumer в цикле проверяет очередь — высокий CPU, плохая масштабируемость.



Решение №1 — Condition variable

CONDITION ПОЗВОЛЯЕТ ЖДАТЬ ПОЯВЛЕНИЯ ДАННЫХ. Модель: mutex + condition. Consumer захватывает mutex, проверяет очередь; если пусто — вызывает `wait(condition, mutex)`, ядро освобождает mutex и блокирует поток. Producer после `push()` вызывает `notify / notify_all`, пробуждённый Consumer снова захватывает mutex и забирает данные.

Condition variable: Пример кода

```
import threading
from collections import deque

queue = deque()
mutex = threading.Lock()
cond = threading.Condition(mutex)

def consumer():
    with cond:
        while not queue:
            cond.wait() # освобождает mutex и блокирует поток
        item = queue.popleft()
    return item

def producer(item):
    with cond:
        queue.append(item)
        cond.notify() # будит одного Consumer
```





Решение №2 — Semaphore (счётный)

Идея: использовать семафор для отслеживания числа доступных элементов. Обычно применяют два семафора: `items` (количество занятых слотов) и `spaces` (количество свободных слотов) — плюс `mutex` для атомарного доступа к очереди при `push/pop`.

Semaphore: Пример кода

```
import threading
from collections import deque

CAP = 10
queue = deque()

items = threading.Semaphore(0) # изначально 0 элементов
spaces = threading.Semaphore(CAP) # CAP – ёмкость буфера
mutex = threading.Lock()

# Producer
def producer(item):
    spaces.acquire()           # ждёт свободный слот
    with mutex:
        queue.append(item)
    items.release()           # увеличивает число доступных элементов

# Consumer
def consumer():
    items.acquire()           # ждёт доступный элемент
    with mutex:
        item = queue.popleft()
    spaces.release()         # освобождает слот
    return item
```



Сравнение: Condition vs Semaphore

Condition

Предназначен для ожидания события (сигнала «появились данные»). Требуется `mutex` при проверке состояния. Позволяет явно пробудить одного или всех ожидающих.

Semaphore

Счётный ресурсный примитив. Хорош для подсчёта элементов/слотов и контроля пропускной способности. Может использоваться без внешнего `mutex` для простых счётчиков, но при доступе к сложным структурам всё равно нужен `mutex`.



Пример, где без condition нельзя эффективно



Сценарий: Consumer ожидает редких и нерегулярных событий (например, поток данных от сетевого устройства с большими паузами). Busy-wait или частый polling убивает CPU и повышает задержки. Семафор можно использовать только если Producer всегда увеличивает счётчик — но если событие определяется сложным предикатом (несколько полей, приоритеты, временные окна), семафор перестаёт отражать семантику события.

Эффективный подход: condition variable

```
import threading

queue = []
mutex = threading.Lock()
condition = threading.Condition(mutex)

def consumer():
    with condition:
        while not queue:
            condition.wait()
        item = queue.pop(0)
        return item

def producer(item):
    with condition:
        queue.append(item)
        condition.notify()
```





Неэффективный подход: busy-wait

```
import threading
import time

queue = []
mutex = threading.Lock()

while True:
    with mutex:
        if queue:
            item = queue.pop(0)
            break
    time.sleep(0.001) # временная оптимизация — всё
                     # равно неэффективно при редких событиях
```